REST Principles

resources, verbs, HATEOAS and "cancel order"

Watch the 4-min video

In one line: REST means resources named by URIs, standard HTTP methods as the verbs, and a set of constraints (stateless, cacheable, layered, uniform interface); "JSON over HTTP" alone is not REST.

1 Resources are nouns, methods are verbs

Almost every company calls its API RESTful. Many just mean "JSON over HTTP". Interviewers know the difference, so learn what REST actually requires.

REST is built around **resources**: the nouns of your domain, such as users, orders and products. Each resource has a **URI**, like `/users/42`. You do not put verbs in the path (`/getUser`, `/createOrder`). The **HTTP method is the verb**.

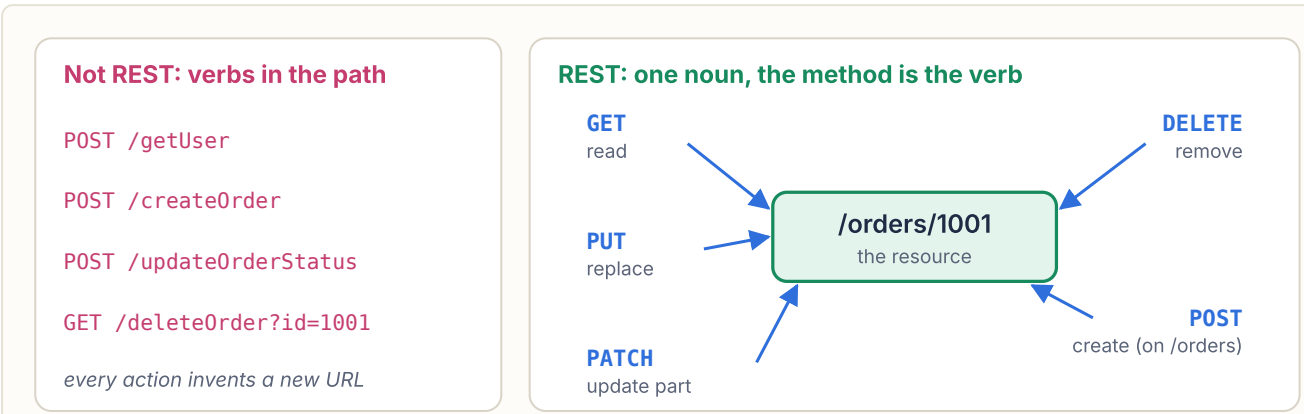


Figure 1. Left: an RPC-style API invents a URL per action. Right: one resource URI, and the standard methods say what to do with it.

Method	Meaning	Safe?	Idempotent?
`GET`	Read a resource or a collection	Yes	Yes
`POST`	Create a new item in a collection (or run an action)	No	No, unless you add a key
`PUT`	Replace the whole resource	No	Yes
`PATCH`	Update some fields	No	Not guaranteed
`DELETE`	Remove the resource	No	Yes

Safe means it changes nothing on the server. **Idempotent** means sending it twice has the same effect as sending it once. This matters because networks retry.

2 The architectural constraints

REST (from Roy Fielding's 2000 dissertation) is defined by constraints, not by JSON or URLs:



Figure 2. The REST constraints. Most "RESTful" APIs follow 1–5; the hypermedia part of the uniform interface (6) is the one usually skipped.

Two of them carry most of the weight in system design. **Stateless**: no session lives in server memory, so any server behind the load balancer can answer any request. **Cacheable**: headers like `Cache-Control` and `ETag` let browsers, CDNs and proxies (the **layered** part) serve repeat reads without touching your servers.

3 HATEOAS: the constraint everyone skips

HATEOAS stands for *hypermedia as the engine of application state*. Responses include **links to what the client can do next**. A pending order carries a cancel link; a shipped order does not. The client follows links instead of building URLs itself.

```
{ "id": 1001,
  "status":    "pending",
  "_links": {
    "self": "/orders/1001"
    "pay": "/orders/1001/payment"
    "cancel": "/orders/1001:cancel"
} }
```

```
{ "id": 1002,
  "status":    "shipped",
  "_links": {
    "self": "/orders/1002"
    "track": "/orders/1002/tracking"
} }
```

no cancel link: too late to cancel

Figure 3. The same resource type, two states. The server, not the client, decides which actions are available right now.

It is elegant, but most clients hardcode URLs anyway. So few public APIs go all the way, and that is fine for most teams. In an interview, say that you know it is part of the full definition, and explain why you would or would not use it.

4 Designing the e-commerce endpoints

Endpoint	What it does
`GET /users/42/orders`	List one user's orders
`POST /users/42/orders`	Create an order for that user (returns `201 Created` + `Location`)
`GET /orders/1001`	Fetch an order directly: orders deserve their own top-level path
`GET /orders?status=shipped`	Filter a collection with query parameters

The rules: **plural nouns**, **lowercase with hyphens** (`/shipping-addresses`), **nesting at most one level**, and **query parameters for filtering**.

5 How to model “cancel order”

Cancel is not plain CRUD, so it is the classic interview question. Three options:

Option	Trade-off
`PATCH /orders/1001` body `{"status":"cancelled"}`	Simple, but refunds and restocking hide behind a field update, and clients may try to set any status.
`POST /orders/1001/cancellation`	Treat the action as a resource: a cancellation is created, and can record who and why.
`POST /orders/1001:cancel`	A **custom method**, the style in Google's API design guidelines. Clear intent, one POST.

Whichever you pick, use status codes properly: **200** with the updated order, or **202 Accepted** if the refund runs asynchronously; **409 Conflict** if it already shipped; **404** if the order does not exist. And make it **idempotent**: cancelling twice must not refund twice.

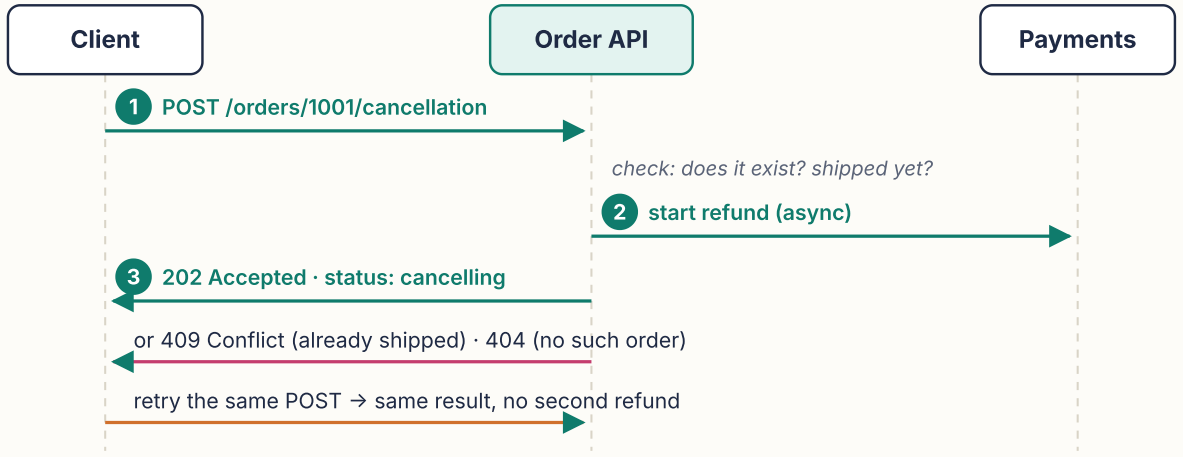


Figure 4. Cancel as a sub-resource. The API checks the business rules, starts the refund, and answers 202; a retry finds the existing cancellation and returns the same result.

WORKED EXAMPLE

```
POST /orders/1001/cancellation
Authorization: Bearer eyJ...
Idempotency-Key: 7f3c-91ab
{ "reason": "ordered by mistake" }

→ 202 Accepted
{ "order_id": 1001, "status": "cancelling", "refund": "pending" }
```

The client's mobile network drops before the reply arrives, so it retries with the same `Idempotency-Key`. The server sees the key (or the existing cancellation for order 1001) and returns the same 202. One refund, not two. If the order had shipped an hour earlier, the first call returns `409 Conflict` with a message such as “order already shipped; request a return instead”.

COMMON MISTAKES

- Verbs in paths: `/getOrders`, `/deleteUser?id=5`.
- Returning `200 OK` with `{"error": ...}` in the body instead of a real 4xx/5xx code.
- Storing session state in server memory, which breaks statelessness and load balancing.
- Making cancel a status PATCH, so a retry or a bad client can trigger a second refund or set "shipped".

6 Key terms

Resource: a noun of the domain, named by a URI

Uniform interface: same methods, codes, representations

Custom method: POST /orders/1001:cancel

URI: the resource's address, e.g. /orders/1001

HATEOAS: responses link to next actions

409 Conflict: request clashes with current state

Stateless: each request carries all its context

Idempotent: repeat = same effect as once

202 Accepted: accepted, finishing asynchronously

7 Interview questions

Q1. What makes an API RESTful? BASIC

Resources identified by URIs; standard HTTP methods as the verbs; stateless requests; cacheable responses; a layered system; and a uniform interface, with hypermedia links (HATEOAS) as the full ideal. JSON over HTTP with verb URLs is not REST.

Q2. Which HTTP methods are idempotent? BASIC

GET, PUT and DELETE (and HEAD, OPTIONS). Deleting /orders/1001 twice leaves it deleted. POST is not: two POSTs to /orders make two orders, unless you add an idempotency key. PATCH is not guaranteed either.

Q3. What does "stateless" mean, and why does it help scaling? BASIC

Each request carries its own context (auth token, IDs), so no server needs memory of earlier requests. Any of, say, 20 servers behind the load balancer can handle any request, and you can add a 21st without moving sessions.

Q4. How would you model "cancel order" in REST? SCENARIO

As a sub-resource, POST /orders/1001/cancellation, or a custom method, POST /orders/1001:cancel, not a hidden status PATCH. Return 200 (or 202 if the refund is async), 409 if it already shipped, 404 if missing, and make it idempotent.

Q5. What is HATEOAS, and why do so few APIs use it? TRICKY

Responses carry links to valid next actions, so a pending order shows a cancel link and a shipped one does not. Clients in practice hardcode URLs and SDKs, so the benefit rarely pays for the extra payload and tooling; most teams stop at the other constraints.

Q6. A client cancels, the response times out, and it retries. What must happen? TRICKY

The second call must not issue a second refund. Detect the existing cancellation (or the same Idempotency-Key) and return the same result, 202 or 200. Only a different, conflicting request should get 409.

Next: SD-C-002 · GraphQL for System Design backendroadmap.com